Modül 27

File Upload & Download

Multipart, Progress, Chunked

Modül:	27 / 30
Ders Sayısı:	7 ders
Seviye:	Orta-İleri
Versiyon:	Angular v21

Hazırlayan: Claude • Türkçe Angular v21 Eğitimi

🎉 Modül 27'ye Hoş Geldin

Dosya yönetimi — TSE'nin Flexi.Belge sistemi gibi document management uygulamalarında çok kritik. Bu modülde upload progress tracking, chunked upload, streaming download, ve drag-drop UI'larını işliyoruz.

Öğrenecekler:

- ✓ Multipart form data ile upload
- ✓ Upload progress tracking
- ✓ Multiple file upload
- ✓ Drag & drop UI
- ✓ Chunked upload (büyük dosyalar)
- ✓ File download with progress
- ✓ Streaming download (Blob)

DERS 27.1

Basic File Upload

FormData ile dosya yükleme.

📤 Single File Upload

```
@Injectable({ providedIn: 'root' })
export class FileUploadService {
  private http = inject(HttpClient);

  uploadFile(file: File): Observable<UploadResult> {
    const formData = new FormData();
    formData.append('file', file);

    return this.http.post<UploadResult>('/api/files/upload', formData);
  }

  uploadWithMetadata(file: File, metadata: any): Observable<UploadResult> {
    const formData = new FormData();
    formData.append('file', file);
    formData.append('metadata', JSON.stringify(metadata));

    return this.http.post<UploadResult>('/api/files/upload', formData);
  }
}
```

🎯 Component Kullanımı

```
@Component({
  template: `
    <input
      type="file"
      (change)="onFileSelect($event)"
      accept=".pdf,.docx,image/*">

    @if (selectedFile()) {
      <p>{{ selectedFile()?.name }} ({{ selectedFile()?.size | fileSize }})</p>
      <button (click)="upload()">Yükle</button>
    }
  `
})
export class UploaderComponent {
  private uploadService = inject(FileUploadService);
  selectedFile = signal<File | null>(null);

  onFileSelect(event: Event) {
    const input = event.target as HTMLInputElement;
    if (input.files && input.files[0]) {
      this.selectedFile.set(input.files[0]);
    }
  }

  upload() {
    const file = this.selectedFile();
    if (!file) return;

    this.uploadService.uploadFile(file)
      .pipe(takeUntilDestroyed())
      .subscribe({
        next: result => console.log('Uploaded:', result),
        error: err => console.error('Failed:', err)
      });
  }
}
```

DERS 27.2

Upload Progress Tracking

Yüklenme yüzdesini kullanıcıya göstermek.

HttpEvent Tracking

```
import { HttpClient, HttpEventType, HttpProgressEvent } from '@angular/common/http';

export interface UploadProgress {
  type: 'progress' | 'response';
  progress?: number;
  body?: any;
}

@Injectable({...})
export class UploadService {
  private http = inject(HttpClient);

  upload(file: File): Observable<UploadProgress> {
    const formData = new FormData();
    formData.append('file', file);

    return this.http.post('/api/upload', formData, {
      reportProgress: true,
      observe: 'events'
    }).pipe(
      map(event => {
        if (event.type === HttpEventType.UploadProgress) {
          const progressEvent = event as HttpProgressEvent;
          const progress = Math.round(
            100 * progressEvent.loaded / (progressEvent.total || 1)
          );
          return { type: 'progress', progress } as UploadProgress;
        }

        if (event.type === HttpEventType.Response) {
          return { type: 'response', body: event.body } as UploadProgress;
        }

        return null;
      }),
      filter((e): e is UploadProgress => e !== null)
    );
  }
}
```

Progress UI

```
@Component({
  template: `
    <input type="file" (change)="onFile($event)">

    @if (uploading()) {
      <div class="progress-wrapper">
        <div class="progress-bar" [style.width.%]="progress()"></div>
        <span>{{ progress() }}%</span>
      </div>
    }

    @if (uploadResult()) {
      <p class="success">✓ Yüklendi</p>
    }
  `
})
export class ProgressUploaderComponent {
  private uploadService = inject(UploadService);
  uploading = signal(false);
  progress = signal(0);
  uploadResult = signal<any>(null);

  onFile(event: Event) {
    const file = (event.target as HTMLInputElement).files?.[0];
    if (!file) return;

    this.uploading.set(true);
    this.progress.set(0);

    this.uploadService.upload(file)
      .pipe(takeUntilDestroyed())
      .subscribe({
        next: event => {
          if (event.type === 'progress') {
            this.progress.set(event.progress!);
          } else if (event.type === 'response') {
            this.uploading.set(false);
            this.uploadResult.set(event.body);
          }
        },
        error: () => this.uploading.set(false)
      });
  }
}
```

DERS 27.3

Multiple File Upload

Birden fazla dosya — paralel veya sıralı.

📁 Multiple Files — Tek FormData

```
uploadMultiple(files: File[]): Observable<UploadProgress> {
  const formData = new FormData();
  files.forEach(file => formData.append('files', file));

  return this.http.post('/api/upload-multiple', formData, {
    reportProgress: true,
    observe: 'events'
  }).pipe(
    map(event => {
      if (event.type === HttpEventType.UploadProgress) {
        const progress = Math.round(100 * event.loaded / (event.total || 1));
        return { type: 'progress', progress };
      }
      if (event.type === HttpEventType.Response) {
        return { type: 'response', body: event.body };
      }
      return null;
    })
  );
}
```

⚡ Parallel Upload (Per File Progress)

```
import { forkJoin, from, mergeMap } from 'rxjs';

uploadFilesParallel(files: File[]): Observable<FileUploadProgress[]> {
  const uploads$ = files.map((file, idx) =>
    this.upload(file).pipe(
      map(event => ({ fileIndex: idx, ...event }))
    )
  );

  return merge(...uploads$);
}

interface FileUploadProgress {
  fileIndex: number;
  type: 'progress' | 'response';
  progress?: number;
}
```

🖼️ Multi-File UI — Template

```
@Component({
  template: `
    <input type="file" multiple (change)="onFiles($event)">

    @for (item of fileQueue(); track item.id) {
      <div class="file-item">
        <span>{{ item.file.name }}</span>
        <div class="progress" [style.width.%]="item.progress"></div>
        @switch (item.status) {
          @case ('pending') { <span>Bekliyor</span> }
          @case ('uploading') { <span>{{ item.progress }}%</span> }
          @case ('done') { <span>✓</span> }
          @case ('error') { <span>✗</span> }
        }
      </div>
    }
  `
})
export class MultiUploaderComponent {
  private uploadService = inject(UploadService);
  fileQueue = signal<UploadItem[]>([]);
  // ... method'lar sonraki bloğta
}
```

🖼️ Multi-File UI — Logic

```
  onFiles(event: Event) {
    const files = Array.from((event.target as HTMLInputElement).files || []);

    const items: UploadItem[] = files.map((file, i) => ({
      id: crypto.randomUUID(),
      file,
      progress: 0,
      status: 'pending'
    }));

    this.fileQueue.set(items);
    this.uploadAll();
  }

  uploadAll() {
    this.fileQueue().forEach(item => this.uploadOne(item));
  }

  uploadOne(item: UploadItem) {
    this.updateItem(item.id, { status: 'uploading' });

    this.uploadService.upload(item.file)
      .pipe(takeUntilDestroyed())
      .subscribe({
        next: event => {
          if (event.type === 'progress') {
            this.updateItem(item.id, { progress: event.progress });
          } else {
            this.updateItem(item.id, { status: 'done', progress: 100 });
          }
        },
        error: () => this.updateItem(item.id, { status: 'error' })
      });
  }

  private updateItem(id: string, updates: Partial<UploadItem>) {
    this.fileQueue.update(q =>
      q.map(item => item.id === id ? { ...item, ...updates } : item)
    );
  }
```

DERS 27.4

Drag & Drop UI

Modern drag-drop file upload deneyimi.

Drop Zone Directive

```
@Directive({
  selector: '[appDropZone]',
  standalone: true,
  host: {
    '(dragover)': 'onDragOver($event)',
    '(dragleave)': 'onDragLeave($event)',
    '(drop)': 'onDrop($event)',
    '[class.drag-active]': 'isDragging()',
  }
})
export class DropZoneDirective {
  filesDropped = output<File[]>();
  isDragging = signal(false);

  onDragOver(event: DragEvent) {
    event.preventDefault();
    event.stopPropagation();
    this.isDragging.set(true);
  }

  onDragLeave(event: DragEvent) {
    event.preventDefault();
    event.stopPropagation();
    this.isDragging.set(false);
  }

  onDrop(event: DragEvent) {
    event.preventDefault();
    event.stopPropagation();
    this.isDragging.set(false);

    const files = Array.from(event.dataTransfer?.files || []);
    if (files.length > 0) {
      this.filesDropped.emit(files);
    }
  }
}
```

Drop Zone Component

```
@Component({
  selector: 'app-drop-zone',
  imports: [DropZoneDirective],
  template: `
    <div
      appDropZone
      (filesDropped)="onFilesDropped($event)"
      class="drop-zone">

      <p>📂 Dosyaları buraya sürükle</p>
      <p>veya</p>

      <input
        #fileInput
        type="file"
        multiple
        hidden
        (change)="onSelect($event)">

      <button (click)="fileInput.click()">Dosya Seç</button>
    </div>
  `,
  styles: [`
    .drop-zone {
      border: 2px dashed #cbd5e1;
      border-radius: 8px;
      padding: 40px;
      text-align: center;
      transition: all 200ms ease;
    }

    .drop-zone.drag-active {
      border-color: #4f46e5;
      background: #eef2ff;
      transform: scale(1.02);
    }
  `]
})
export class DropZoneComponent {
  filesAdded = output<File[]>();

  onFilesDropped(files: File[]) {
    this.filesAdded.emit(files);
  }

  onSelect(event: Event) {
    const files = Array.from((event.target as HTMLInputElement).files || []);
    this.filesAdded.emit(files);
  }
}
```

DERS 27.5

Chunked Upload (Büyük Dosyalar)

100MB+ dosyalar için parça parça upload.

❓ Neden Chunked?

- ✓ Büyük dosya: server'ın tek seferde alamayacağı boyut
- ✓ Resume: bağlantı kopsa kaldığı yerden devam
- ✓ Memory: server'ın tüm dosyayı RAM'de tutmaması
- ✓ Progress: daha hassas progress reporting

⚙ Chunked Service

```
@Injectable({...})
export class ChunkedUploadService {
  private http = inject(HttpClient);
  private CHUNK_SIZE = 5 * 1024 * 1024;  // 5 MB

  async uploadInChunks(file: File): Promise<string> {
    const totalChunks = Math.ceil(file.size / this.CHUNK_SIZE);
    const uploadId = await this.startUpload(file);

    for (let chunk = 0; chunk < totalChunks; chunk++) {
      const start = chunk * this.CHUNK_SIZE;
      const end = Math.min(start + this.CHUNK_SIZE, file.size);
      const blob = file.slice(start, end);

      await this.uploadChunk(uploadId, chunk, blob);
    }

    return await this.completeUpload(uploadId);
  }

  private async startUpload(file: File): Promise<string> {
    const res = await firstValueFrom(
      this.http.post<{ uploadId: string }>('/api/upload/start', {
        filename: file.name,
        size: file.size,
        type: file.type
      })
    );
    return res.uploadId;
  }

  private async uploadChunk(uploadId: string, chunkIndex: number, blob: Blob):
Promise<void> {
    const formData = new FormData();
    formData.append('chunk', blob);
    formData.append('chunkIndex', chunkIndex.toString());
    formData.append('uploadId', uploadId);

    await firstValueFrom(
      this.http.post('/api/upload/chunk', formData)
    );
  }

  private async completeUpload(uploadId: string): Promise<string> {
    const res = await firstValueFrom(
      this.http.post<{ fileUrl: string }>('/api/upload/complete', { uploadId })
    );
    return res.fileUrl;
  }
}
```

🔄 Resumable Upload

```
async uploadResumable(file: File, uploadId: string) {
  // 1. Resume noktasını bul
  const status = await firstValueFrom(
    this.http.get<{ uploadedChunks: number[] }>(`/api/upload/${uploadId}/status`)
  );

  const totalChunks = Math.ceil(file.size / this.CHUNK_SIZE);

  // 2. Eksik chunk'ları yükle
  for (let i = 0; i < totalChunks; i++) {
    if (status.uploadedChunks.includes(i)) {
      continue;  // Zaten yüklenmiş, atla
    }

    const start = i * this.CHUNK_SIZE;
    const end = Math.min(start + this.CHUNK_SIZE, file.size);
    const blob = file.slice(start, end);

    await this.uploadChunk(uploadId, i, blob);
  }

  return await this.completeUpload(uploadId);
}
```

DERS 27.6

File Download with Progress

Server'dan dosya indirme.

⬇ Basic Download

```
@Injectable({...})
export class DownloadService {
  private http = inject(HttpClient);

  download(url: string, filename: string) {
    this.http.get(url, { responseType: 'blob' }).subscribe(blob => {
      this.saveBlob(blob, filename);
    });
  }

  private saveBlob(blob: Blob, filename: string) {
    const url = window.URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url;
    a.download = filename;
    document.body.appendChild(a);
    a.click();
    document.body.removeChild(a);
    window.URL.revokeObjectURL(url);
  }
}
```

□ Download with Progress

```
downloadWithProgress(url: string): Observable<DownloadProgress> {
  return this.http.get(url, {
    responseType: 'blob',
    reportProgress: true,
    observe: 'events'
  }).pipe(
    map(event => {
      if (event.type === HttpEventType.DownloadProgress) {
        return {
          type: 'progress',
          progress: Math.round(100 * event.loaded / (event.total || 1))
        };
      }
      if (event.type === HttpEventType.Response) {
        return { type: 'complete', blob: event.body as Blob };
      }
      return null;
    }),
    filter((e): e is DownloadProgress => e !== null)
  );
}
```

□ Save Blob (Browser-safe)

```
import { saveAs } from 'file-saver';

// Library kullan
saveAs(blob, 'document.pdf');

// Manuel
function saveBlob(blob: Blob, filename: string) {
  // IE/Edge için
  if ('msSaveBlob' in navigator) {
    (navigator as any).msSaveBlob(blob, filename);
    return;
  }

  const url = window.URL.createObjectURL(blob);
  const a = document.createElement('a');
  a.href = url;
  a.download = filename;
  a.style.display = 'none';
  document.body.appendChild(a);
  a.click();
  document.body.removeChild(a);

  // Cleanup
  setTimeout(() => window.URL.revokeObjectURL(url), 1000);
}
```

DERS 27.7

Streaming Download

Büyük dosyalar için memory-efficient download.

Fetch Streaming

```
async function streamDownload(url: string, filename: string,
                              onProgress: (loaded: number, total: number) => void) {
  const response = await fetch(url);

  if (!response.ok) throw new Error('Download failed');

  const contentLength = response.headers.get('content-length');
  const total = parseInt(contentLength || '0', 10);

  const reader = response.body!.getReader();
  const chunks: Uint8Array[] = [];
  let loaded = 0;

  while (true) {
    const { done, value } = await reader.read();
    if (done) break;

    chunks.push(value);
    loaded += value.length;
    onProgress(loaded, total);
  }

  const blob = new Blob(chunks);
  saveBlob(blob, filename);
}
```

Use Case: Large Report Download

```
@Component({
  template: `
    <button (click)="downloadReport()" [disabled]="downloading()">
      @if (downloading()) {
        İndiriliyor... {{ progress() }}%
      } @else {
        Rapor İndir (150 MB)
      }
    </button>
  `
})
export class ReportDownloadComponent {
  downloading = signal(false);
  progress = signal(0);

  async downloadReport() {
    this.downloading.set(true);
    this.progress.set(0);

    try {
      await streamDownload(
        '/api/reports/annual.pdf',
        'annual-report-2026.pdf',
        (loaded, total) => {
          this.progress.set(Math.round(100 * loaded / total));
        }
      );
    } finally {
      this.downloading.set(false);
    }
  }
}
```

🔐 Auth ile Download

```
downloadSecure(fileId: string) {
  // Token header ile (HttpClient auto-adds)
  this.http.get(`/api/files/${fileId}/download`, {
    responseType: 'blob',
    reportProgress: true,
    observe: 'events',
    headers: { Authorization: `Bearer ${this.token}` }
  }).subscribe(event => {
    if (event.type === HttpEventType.Response) {
      // Filename from Content-Disposition header
      const disposition = event.headers.get('Content-Disposition');
      const filename = this.extractFilename(disposition) || 'download';
      saveBlob(event.body as Blob, filename);
    }
  });
}

private extractFilename(disposition: string | null): string | null {
  if (!disposition) return null;
  const match = /filename[^;=\n]*=(['"]).*?\2|[^;\n]*)/.exec(disposition);
  return match?.[1]?.replace(/['"]/g, '') || null;
}
```

Modül 27 Özeti

File handling profesyonel seviyede. Upload, download, progress, chunked, streaming — Flexi.Belge gibi document management uygulamaların temelleri.

Neler Öğrendin?

- ✓ Multipart form data ile upload
- ✓ Upload progress tracking
- ✓ Multiple file upload
- ✓ Drag & drop UI
- ✓ Chunked upload (büyük dosyalar)
- ✓ Resumable upload
- ✓ Download progress
- ✓ Streaming download (büyük dosya)
- ✓ Secure download with auth

Sıradaki Modül

Modül 28 — Authentication & Authorization. Login flow, JWT, refresh, role-based access, route guards.